Université Grenoble Alpes
L3 Informatique

Année 2026-2027

Architectures logicielles et matérielles
Semaine 1 TD 1

L'objectif de ce TD est de réviser les codages des nombres entiers non signés et signés et quelques bases d'arithmétique.

Questions à traiter en début de séance. (remise à 8h15)

Question 1. *Représentation des entiers non signés*
On rappelle le codage des **entiers non signés** en base B sur n digits :

$$x_{n-1}\ x_{n-2}\ \ldots\ x_2\ x_1\ x_0$$

digit de poids fort — digit de poids faible

où chaque digit x_k peut valoir :
- en base B=2 (binaire) : 0 ou 1. Dans ce cas les digits sont appelés des "bits" (binary digit), et un "octet" est une suite de 8 bits.
- en base B=8 (octal) : 0, 1, 2,… ou 7
- en base B=16 (hexadécimal) : 0, 1, 2,…, 8, 9, A, B, C, D, E, F

avec l'interprétation : $X = \sum_{k=0}^{n-1} x_k\, B^k$

• Rappeler le **principe** permettant de passer aisément d'une représentation en **binaire** à une représentation en **hexadécimal** et vice-versa (sans passer par le décimal)

• **Illustrer** ce principe sur les **exemples** suivants (détailler les explications) :

Valeur hexadécimale de **01010010** :
Valeur binaire de **7A** :

Question 2. *Opérations bit à bit, masquages*
On rappelle que, outre les opérateurs logiques présents dans tous les langages de programmation (notés comme suit en C : et logique &&, ou logique ||, non logique !), C propose des **opérateurs bit à bit**. On les utilisera sur des entiers non signés, et on prendra garde à ne pas les confondre avec les opérateurs logiques.

Avec ces opérateurs, chaque bit de l'entier résultant correspond à l'opération booléenne sur les bits du même rang des deux opérandes.
Il s'agit du *et bit à bit* **&**, du *ou bit à bit* **|**, du complément à 1 (*négation bit à bit*) **~**, et du *ou exclusif bit à bit* **^**

Des **opérateurs de décalage** sont également présents :

- *décalage logique à gauche* (logic shift left) de k bits, **x << k** : les bits de x sont décalés de k positions sur la gauche, et insertion de k bits de valeur 0 à droite
- *décalage logique à droite* (logic shift right) de k bits, **x >> k** : les bits de x sont décalés de k positions sur la droite, et insertion de k bits de valeur 0 à gauche.

NOTE. Il sera nécessaire, dans toutes les séances à venir, de caractériser correctement la taille, et l'interprétation signée/non signée, des valeurs manipulées. On utilisera pour cela les **types** définis dans *stdint.h* :

uint8_t = entier non signé sur 1 octet, uint16_t = entier non signé sur 2 octets,…
int8_t = entier signé sur 1 octet, int16_t = entier signé sur 2 octets,…

• Faire les **calculs** :

```
    10101010                    10101010
&   00100000            &       00000100
    --------                    --------
```

Que pouvez-vous en conclure quant à la notion de masquage ?

• En vous aidant de ces réponses, rappeler le **principe** selon lequel il est possible de **consulter la valeur du bit numéro k** dans un vecteur de bits (nombre entier naturel).

• Donner **l'expression en C** permettant de calculer la valeur 0000000000100000 (sur 16 bits)

Enfin, on rappelle également les bases du **codage en complément à 2 des entiers signés** (entiers relatifs) en binaire sur n bits :
Pour un entier X positif, la représentation de l'entier négatif -X correspond à $\overline{X} + 1$
c'est à dire la négation bit à bit de X (i.e., complément à 1) + 1.

NB. En pratique il suffit, en partant de la droite, de conserver tous les 0 jusqu'au premier 1 (compris) et d'inverser ensuite tous les bits jusqu'au bit de poids fort.

Ceci revient à calculer la représentation de l'entier naturel 2^n - X (par exemple, la représentation de -3 sur 4 bits correspond à la représentation de 16 - 3 = 13). Autrement dit, coder l'entier $x_{n-1}\ x_{n-2}\ \ldots\ x_1\ x_0$ négatif reviendrait à coder l'entier non signé :

$$X = 2^n - \sum_{k=0}^{n-1} x_k\ B^k$$

Exemples :

binaire	interp. non signée	interp. en complément à 2
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

Exercice 1.

Q1. Combien de valeurs différentes peut-on coder en binaire sur N bits ? (quelle que soit l'interprétation)

Q2. Quel est l'intervalle de nombres entiers naturels que l'on peut coder en binaire sur N bits ?

Q3. Quel est l'intervalle de nombres entiers relatifs que l'on peut coder en binaire sur N bits ?

Q4. Donner les représentations en complément à 2 sur un octet des nombres entiers 44 et -44

Q5. Rappeler la représentation de l'entier naturel 14 sur 4 bits. Même question pour l'entier naturel 7.
Calculer la somme en binaire de ces deux nombres entiers naturels. Qu'observez-vous ? Comment se nomme l'indicateur qui est apparu et que signifie-t-il ?

Exercice 2.

Faisons tout d'abord quelques rappels concernant arithmétique et indicateurs (flags).
Nous venons de rappeler, dans la dernière question de l'exercice 1, l'existence d'indicateurs (**flags**) qui fournissent des indications sur les caractéristiques du résultat d'un calcul. Nous avons revu le flag **C** ("Carry") qui indique que le résultat de l'opération (sur entiers naturels) est trop grand pour être représentable sur le nombre de bits prévu.

Plus généralement, nous allons rappeler l'existence de 4 flags typiques :

- **C (Carry)** : le résultat est non représentable sur le nombre de bits prévu - *C est la retenue sortante*
- **V (oVerflow)** : le signe du résultat est le signe opposé à celui du résultat apparent (apparaît comme négatif mais en réalité est positif, ou l'inverse) - *V est à 1 si les deux dernières retenues sont différentes*
- **N (Negative)** : le résultat apparent est négatif - *N est le bit de poids fort du résultat apparent*
- **Z (Zero)** : le résultat est nul - *Z est à 1 si tous les bits du résultat sont à 0*

Ces 4 indicateurs, ZNCV, peuvent jouer un rôle fondamental pour l'évaluation des **conditions de branchements** dans les programmes (conditionnelle *if*, boucle *for*,…) sans avoir recours à des opérations de comparaisons.
Voyons ici quelques cas emblématiques (on suppose ci-dessous que a et b sont deux entiers relatifs) :

- la condition d'une instruction conditionnelle ou itérative est de la forme (**a == b**) → pour l'évaluer, on calcule a-b et on inspecte la valeur du flag **Z** (doit être à 1)
- la condition d'une instruction conditionnelle ou itérative est de la forme (**a != b**) → pour l'évaluer, on calcule a-b et on inspecte la valeur du flag **Z** (doit être à 0)
- la condition d'une instruction conditionnelle ou itérative est de la forme (**a < b**) → pour l'évaluer, on calcule a-b et on inspecte la valeur de **V xor N** (doit être à 1) - en effet, le résultat de a-b doit être négatif, ou bien paraître positif mais avec un signe apparent incorrect.

Q1. Convertir en binaire sur un octet les nombres hexadécimaux 7A et 52.
Calculer la somme de ces deux nombres binaires signés, et les flags ZNCV pour cette addition. Interpréter avec soin les valeurs de ces flags.

Q2. Convertir en binaire sur un octet les nombres hexadécimaux 2A et 52.
Calculer la somme de ces deux nombres binaires signés, et les flags ZNCV pour cette addition. Interpréter avec soin les valeurs de ces flags.

Q3. Convertir en binaire sur un octet les nombres hexadécimaux AC et 8A.
Calculer la somme de ces deux nombres binaires signés, et les flags ZNCV pour cette addition. Interpréter avec soin les valeurs de ces flags.

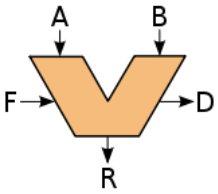
Q4. Convertir en binaire sur un octet les nombres hexadécimaux AC et 72.
Calculer la somme de ces deux nombres binaires signés, et les flags ZNCV pour cette addition. Interpréter avec soin les valeurs de ces flags.

Q5. Convertir en binaire sur 4 bits les entiers 6 et -8. Calculer la somme de ces deux nombres binaires signés, et les flags ZNCV pour cette addition.
Qu'en déduisez-vous pour l'évaluation de la condition **a < b** si a=6 et b=8 ?

Q6. Convertir en binaire sur 4 bits les entiers 6 et -2. Calculer la somme de ces deux nombres binaires signés, et les flags ZNCV pour cette addition.
Qu'en déduisez-vous pour l'évaluation de la condition **a < b** si a=6 et b=2 ?

Note. Le composant matériel appelé *Unité Arithmétique et Logique (UAL)*, présent dans un microprocesseur pour réaliser des calculs simples comme opérations logiques de base,

additions et soustractions, a la faculté de produire ces 4 flags typiques. Cette UAL est généralement schématisée comme ci-dessous :



A et B sont les opérandes pour le calcul, F contrôle le choix de l'opération à effectuer (et, ou, décalage, somme,…), et R est le résultat de l'opération. Mais de plus, l'UAL fournit sur la sortie D les valeurs de ces flags usuels ZNCV vus ci-dessus.

Nous approfondirons ces concepts matériels dans la partie Conception de l'UE.

Nous rappelons également que dans certains microprocesseurs (ex. proc. ARM), ces flags ZNCV sont sauvegardés dans un registre d'état spécifique.